Übungsblatt 7: Timing messen und Beobachtungen auswerten

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Was kostet die Beobachtung? (8 Orientierungspunkte)

Didaktischer Ein-Kern-Fall: Eine Funktion benötigt 18 µs eigene Prozessorarbeit. Danach sendet sie die vier ASCII-Zeichen `ACK\n` bei 230400 bit/s mit acht Datenbits, keiner Parität und einem Stopbit (8N1), ohne Nullterminator. Im vorgegebenen blockierenden Modell verlängert die volle Leitungsdauer die Antwort. Weitere Kosten sind null, die relative Abschlussfrist beträgt 180 µs.

1. Berechnen Sie Bitzahl und reine Leitungsdauer. (2 Punkte)
2. Berechnen Sie Antwortzeit und Fristabweichung. (2 Punkte)
3. Bei direktem Speicherzugriff (DMA) überträgt Hardware die vorbereiteten Bytes. Warum ist die Leitungsdauer dann nicht automatisch eigene CPU-Zeit? Nennen Sie verbleibende Arbeit. (2 Punkte)
4. Formulieren Sie einen Messvertrag für die ursprüngliche Funktion ohne Ausgabe im Fenster: Anfang, Ende, Quelle, Einheit und Betriebsbedingungen. Warum kann ein lokaler Zähler trotzdem fremde Arbeit mitzählen? (2 Punkte)

Musterlösung zu Aufgabe 1

1. ASCII kodiert hier jedes Zeichen in einem Byte. `\n` ist ein einzelnes Zeilenumbruchzeichen, also sind es vier Bytes. 8N1 benötigt je Byte ein Startbit, acht Datenbits und ein Stopbit. Damit gilt

$$N_{\text{Bit}} = 4 \cdot (1 + 8 + 1) = 40,$$

$$t_{\text{Leitung}} = \frac{40}{230400}\,\text{s} = 173{,}611\ldots\,\mu\text{s}.$$

2. Nach der Modellannahme wird die volle Dauer zur Arbeit addiert:

$$R = 18 + 173{,}611\ldots = 191{,}611\ldots\,\mu\text{s}.$$

   Die Reserve beträgt $180 - R = -11{,}611\ldots$ µs. Die Frist wird um rund 11,61 µs verletzt. Dies ist eine Modellrechnung zur blockierenden Ausgabe, keine Messung des Boards.

3. Bei DMA kann der Prozessor während der Übertragung andere Arbeit ausführen. Formatierung, Einrichtung des Transfers, Pufferverwaltung und Abschlussbehandlung können trotzdem Prozessorzeit oder Wartezeit kosten. Wann der Job als fertig gilt, entscheidet die Anforderung: Ist die physische Übertragung Teil seines Abschlusses, kann ihre Dauer weiter in der Antwortzeit liegen, obwohl der Prozessor nicht durchgehend für diesen Job arbeitet.
4. Ein möglicher Vertrag lautet: Anfang ist die erste Zählerlesung unmittelbar vor dem Arbeitsabschnitt, Ende die zweite Lesung unmittelbar danach. Quelle ist der fortlaufende lokale Cortex-M7-Zykluszähler, Einheit zunächst Kerntakte. Der konkrete übersetzte Programmstand, tatsächlicher stabiler Kerntakt, laufender Zähler und ein Fenster kürzer als ein Umlauf werden festgehalten. Es gibt keinen Debughalt im Fenster. Gespeichert und angezeigt wird erst danach. Interrupts oder andere Taskausführung zwischen den beiden Lesungen werden mitgezählt, sofern der Zähler weiterläuft. Das Intervall ist daher nicht automatisch reine eigene Prozessorzeit.

Aufgabe 2: Jobantwort und Interruptkette (8 Orientierungspunkte)

Alle folgenden Zeiten sind didaktische Annahmen auf jeweils einer gemeinsamen Uhr. Kontextwechselkosten sind bereits in der Zeitachse enthalten, es gibt keine weitere eigene Arbeit des betrachteten Jobs.

1. Ein Job wird bei 4 ms freigegeben, läuft von 6 bis 9 ms und von 12 bis 14 ms. Er ist bei 14 ms fertig; seine absolute Deadline liegt bei 16 ms. Bestimmen Sie eigene CPU-Zeit, Startverzug, Antwortzeit und Reserve. Zerlegen Sie die Antwortzeit in eigene Arbeit und Zeiten ohne eigene Ausführung. (4 Punkte)
2. Eine Interruptkette hat diese Marken in µs: äußeres Ereignis 200, Eintritt in die Interrupt-Service-Routine (ISR) 207, zugehörige Task wird Ready 214, ISR-Ende 221, Taskstart 229, geforderte Reaktion 242. Bestimmen Sie Eintrittslatenz, ISR-Intervall, Ready-Wartezeit und Gesamtreaktion. Geben Sie eine Zerlegung der Gesamtreaktion ohne überlappende Summanden an. (4 Punkte)

Musterlösung zu Aufgabe 2

1. Die eigene Prozessorzeit summiert nur die laufenden Abschnitte:

$$C = (9 - 6) + (14 - 12) = 5\,\text{ms}.$$

Der Startverzug beträgt $6 - 4 = 2$ ms. Die Antwortzeit reicht dagegen von Freigabe bis Abschluss:

$$R = 14 - 4 = 10\,\text{ms}, \qquad \text{Reserve} = 16 - 14 = 2\,\text{ms}.$$

Die Antwort zerfällt in 2 ms Anfangswarten, 3 ms Unterbrechung eigener Ausführung und 5 ms eigene Arbeit. Es gilt also $R = 2 + 3 + 5 = 10$ ms. Die 5 ms sind die eigene Zeit dieses Modelljobs, keine allgemeine Worst-Case-Ausführungszeit seiner Task.

2. Die Eintrittslatenz beträgt $207 - 200 = 7$ µs. Das ISR-Intervall beträgt $221 - 207 = 14$ µs. Die Task wartet ab Ready bis zu ihrem ersten Start $229 - 214 = 15$ µs. Die Gesamtreaktion beträgt $242 - 200 = 42$ µs. Eine lückenlose, überschneidungsfreie Zerlegung entlang der Uhr ist

$$42 = (207 - 200) + (221 - 207) + (229 - 221) + (242 - 229) = 7 + 14 + 8 + 13\,\mu s.$$

Die Ready-Wartezeit von 214 bis 229 µs enthält bereits die letzten 7 µs des ISR-Intervalls. Man darf deshalb nicht Eintrittslatenz, vollständiges ISR-Intervall, vollständige Ready-Wartezeit und anschließende Arbeit addieren. Ready bedeutet ausführungsbereit, aber noch nicht laufend.

Aufgabe 3: Modulo, Takt und Stichprobe (8 Orientierungspunkte)

1. Ein bestätigter lokaler 32-Bit-Zähler startet bei `0xFFFFFF80` und endet bei `0x00000050`. Berechnen Sie die vorzeichenlose Modulo-Differenz und die Zeit bei stabilen 160 MHz. Wie lange dauert ein voller Umlauf? Welche Annahme über die echte Fensterlänge braucht die Umrechnung? (3 Punkte)
2. Ein Messfenster umfasst 2400 Takte. Rechnen Sie für 160 MHz und für 320 MHz um. Darf ein Cortex-M4-Endwert ohne Zeitkorrelation von einem Cortex-M7-Anfangswert abgezogen werden? (2 Punkte)
3. Fünf vollständige, gültige Antwortintervalle betragen 19, 22, 20, 27 und 21 µs. Bestimmen Sie Minimum, beobachtetes Maximum, Mittelwert und Spanne. Prüfen Sie die Antwortfrist von 25 µs. Welche Aussage erlaubt die Stichprobe über eine Garantie für alle zulässigen Fälle? (3 Punkte)

Musterlösung zu Aufgabe 3

1. Vom Startwert bis zum Umlauf fehlen 128 Takte, danach folgen 80 Takte. Deshalb gilt

$$\Delta c = (c_{\text{Ende}} - c_{\text{Anfang}}) \bmod 2^{32} = 128 + 80 = 208\,\text{Takte},$$

$$\Delta t = \frac{208}{160 \cdot 10^6}\,\text{s} = 1{,}3\,\mu s.$$

Ein voller Umlauf dauert

$$T_{\text{Umlauf}} = \frac{2^{32}}{160 \cdot 10^6}\,\text{s} = 26{,}8435456\,\text{s}.$$

Die echte verstrichene Zeit muss kürzer als dieser Umlauf sein. Gleiche Endstände könnten sonst zusätzlich einen oder mehrere vollständige Umläufe verbergen. Außerdem müssen Takt und Zählerbetrieb über das ganze Fenster zum Vertrag passen.

2. $2400/(160 \cdot 10^6) = 15$ µs, bei 320 MHz sind es 7,5 µs. Die Taktzahl allein legt also noch keine Zeit fest. Zwei lokale Kernzähler haben ohne bestätigte Korrelation weder einen gemeinsamen Nullpunkt noch notwendig dieselbe Skalierung. Ihre Rohwerte dürfen nicht einfach voneinander abgezogen werden.

3. Minimum 19 µs, Maximum 27 µs, Spanne $27 - 19 = 8$ µs. Die Summe beträgt 109 µs, somit ist der Mittelwert $109/5 = 21{,}8$ µs. Der gültige 27-µs-Fall verletzt die Frist um 2 µs. Er widerlegt eine Garantie, die diesen Fall einschließt. Auch ein vollständig fristgerechter endlicher Datensatz würde umgekehrt keine allgemeine Antwortzeitgrenze beweisen. Das beobachtete Antwortmaximum ist außerdem keine WCET: WCET bedeutet Worst-Case Execution Time, obere Grenze der eigenen Ausführungszeit über die definierten zulässigen Fälle.

Aufgabe 4: Ein kurzes Ereignislog beurteilen (8 Orientierungspunkte)

Ein begrenztes Softwarelog erfasst ausgewählte Ereignisse. Seine Zeitstempel stammen von einer bestätigten gemeinsamen Modelluhr in µs. Jobnummern erlauben die Zuordnung von Freigabe zu Abschluss. Eine Verlustmarke bedeutet, dass Ereignisse fehlen können.

Aufgezeichnete Information	Zeit in µs
Job A freigegeben	100
Job A abgeschlossen	126
Job B freigegeben	150
Verlustmarke, danach keine gesicherte Zuordnung	160
Neue Synchronisation und vollständiger Kontext bestätigt	200
Job C freigegeben	210
Job C abgeschlossen	234

1. Welche Antwortintervalle sind auswertbar? Darf für Job B ein fehlender Abschluss mit null oder mit der nächsten Zeitmarke ersetzt werden? (2 Punkte)
2. Nennen Sie den Messvertrag für die auswertbaren Intervalle. Lässt sich aus diesen Freigabe-/Abschlussmarken zusätzlich die eigene CPU-Zeit bestimmen? (2 Punkte)
3. Das Log erzeugt konstant 90000 Ereignisse/s zu acht Byte. Der Abfluss beträgt konstant 540000 Byte/s, anfangs sind 36 KiB frei. Zusatzkosten werden im Modell ignoriert. Nach welcher Zeit ist der freie Puffer gefüllt? Wie wirkt doppelte Kapazität? (2 Punkte)
4. Ein Team schlägt eine Ausgabe nach jedem Ereignis oder einen Breakpoint bei Jobfreigabe vor. Nennen Sie je einen konkreten Beobachtereffekt und eine schonendere Vorgehensweise für die zeitliche Auswertung. (2 Punkte)

Musterlösung zu Aufgabe 4

1. Job A hat die vollständige Antwortzeit $126 - 100 = 26$ µs. Nach bestätigter Synchronisation und bekanntem Kontext ist Job C separat mit $234 - 210 = 24$ µs auswertbar. Für Job B bleibt die Antwort unbekannt. Eine null oder der Zeitpunkt 200 µs würde einen nicht aufgezeichneten Abschluss erfinden. Es wird auch kein Ereignispaar über die Datenlücke gebildet. Die zwei gültigen Fälle haben ein beobachtetes Maximum von 26 µs, das nur diese Fälle beschreibt.

2. Anfang ist die Freigabemarke, Ende die Abschlussmarke desselben Jobs. Die Quelle ist die bestätigte gemeinsame Modelluhr, Einheit µs. Die jeweiligen Ereignispaare liegen in vollständigen Abschnitten mit bekanntem Kontext. Es werden die festgelegten Betriebsbedingungen und der instrumentierte Programmstand dokumentiert. Die Intervalle sind Antwortzeiten. Eigene CPU-Zeit erfordert zusätzliche Information über die tatsächlich ausgeführten Abschnitte und die Zuordnung von Interrupt- beziehungsweise Taskarbeit. Sie folgt nicht aus zwei Endpunkten.

3. 36 KiB entsprechen $36 \cdot 1024 = 36864$ Byte. Der Eingang beträgt $90000 \cdot 8 = 720000$ Byte/s. Der Puffer wächst daher mit $720000 - 540000 = 180000$ Byte/s. Seine Füllzeit ist

$$t_{\text{voll}} = \frac{36864}{180000}\,\text{s} = 0{,}2048\,\text{s}.$$

   Doppelte Kapazität ergibt 0,4096 s. Sie verzögert den Überlauf, beseitigt aber den dauerhaften Ratenüberschuss nicht. Eine begrenzte Ereignisauswahl oder ausreichender Abfluss muss zum geplanten Messfenster passen.

4. Eine Ausgabe benötigt Befehle und kann auf eine Sperre oder Pufferplatz warten. Sie kann dadurch Antwortzeiten und Ausführungsreihenfolgen ändern. Ein Breakpoint hält den untersuchten Kern an, während Peripherie oder der andere Kern weiterlaufen können; danach fehlt möglicherweise der ursprüngliche zeitliche Zusammenhang. Kurze begrenzte Einträge ohne blockierende Ausgabe und eine Auswertung nach Ende des Messfensters reduzieren diese Einflüsse. Ihr Restaufwand wird trotzdem im Messvertrag benannt.

Aufgabe 5: Ein Leerfenster ist kein universeller Abzug

Zusätzliche Verständnisaufgabe, ohne Orientierungspunkte: Leerfenster enthalten 30, 36 und 33 Takte, Arbeitsfenster 480, 560 und 520 Takte. Alle Zahlen sind didaktische Annahmen. Darf von jedem Arbeitsfenster 30 abgezogen und der größte Rest „WCET“ genannt werden? Nennen Sie zwei unabhängige Gründe.

Musterlösung zu Aufgabe 5

Nein. Der kleinste Wert eines getrennten Leerfensters ist nicht nachgewiesen der jeweilige Messaufwand in jedem Arbeitsfenster. Beispielsweise können andere Speicherzustände oder Interrupts die Fenster unterschiedlich beeinflussen. Ein pauschaler Abzug rekonstruiert deshalb nicht automatisch die eigene Prozessorzeit.

Unabhängig davon decken die drei Arbeitsfenster nicht alle zulässigen Eingaben und Systemzustände ab. Der größte rechnerische Rest ist $560 - 30 = 530$ Takte. Diese Zahl ist ein bearbeitetes Stichprobenmaximum, keine bewiesene obere Grenze der Ausführungszeit. Auch ein korrekt bestimmter Messaufwand würde aus endlich vielen günstigen Beobachtungen keinen WCET-Nachweis machen.

Aufgabe 6: Die eigene Zählerauswertung implementieren

Zusätzliche Programmieraufgabe, etwa 30–40 Minuten, ohne Orientierungspunkte. Die vorbereitete Cortex-M7-Basis führt fünf EMPTY- und fünf WORK-Fenster aus. EMPTY enthält keine synthetische Arbeit, WORK die vorgegebene kleine Berechnung. Nach erfolgreichem Lauf schreibt niemand mehr in `lab07_records`. Die Hardwarebasis und der tatsächliche Kerntakt sind vor dem Termin geprüft. Sie implementieren `Lab07_Summarize()` in `summary_student.c` selbst.

1. Prüfen Sie beide Pointer, die Anzahl höchstens `LAB07_CAPACITY` und die Auswahl EMPTY/WORK. Berücksichtigen Sie nur Datensätze der gewählten Art. Ohne passenden Datensatz ist das Ergebnis ungültig, kein künstlicher Nullzeitwert.
2. Bestimmen Sie Anzahl, Minimum, Maximum und 64-Bit-Summe der `cycles_mod32`. Übernehmen Sie `core_hz` aus dem ersten passenden Datensatz. Weisen Sie null oder unterschiedliche Takte innerhalb der gewählten Gruppe zurück.
3. Geben Sie bei Erfolg eins, bei Fehler null zurück. Auf Fehler bleibt die Ausgabe unverändert. Die Basis legt eine erfolgreiche WORK-Auswertung in `lab07_summary` ab; LD1 auf PB0 leuchtet nur bei erfolgreicher Analyse.
4. Rechnen Sie den Mittelwert erst nach dem Versuch mit $\bar{t} = \mathrm{sum}/(\mathrm{count} \cdot f)$ um. Dabei ist f der tatsächliche Kerntakt in Hz. Vergleichen Sie EMPTY und WORK getrennt und erläutern Sie, weshalb das Ergebnis kein Fristnachweis ist.
5. Prüfen Sie die unten genannten Fälle mit selbst angelegten Datensätzen, auf Papier oder nach Ende der Messung im Debugger. Dies sind synthetische Funktionstests, keine erwarteten Boardmesswerte. Halten Sie eigene Implementierung und tatsächlich erreichten Prüfstatus fest.

Synthetischer Prüfinput	Zu prüfende Eigenschaft
Kein Datensatz der gewählten Art	Fehler und unveränderte Ausgabe
Zwei gewählte Datensätze mit verschiedenen `core_hz`	Fehler und unveränderte Ausgabe
Zwei WORK-Werte 3000000000 und 2000000000, beide 160000000 Hz	Richtige Summe oberhalb von 32 Bit
Eine andere Art zwischen zwei WORK-Datensätzen	Nur WORK trägt zur Auswertung bei

Vorbereitung: Projektanleitung, STM32CubeMX und VS Code mit STM32 VS Code Extension für NUCLEO-H755ZI-Q / STM32H755ZIT6. Board und USB/ST-LINK genügen. Es ist kein zusätzlicher Traceadapter erforderlich. Diagnosewerte erst bei beendetem Log und angehaltenem zuständigem Kern lesen. Die Umrechnung setzt einen kontinuierlich laufenden Zähler, stabilen Takt und Fenster kürzer als einen vollständigen Umlauf voraus. Die Auswertungsfunktion kann Verletzungen dieser Annahmen nicht nachträglich erkennen.

Musterlösung zu Aufgabe 6

Die C-Funktion sammelt zuerst in einer lokalen Struktur. Erst wenn alle passenden Datensätze geprüft sind, wird diese Struktur veröffentlicht. So bleibt `*out` bei jedem Fehler unverändert. Ein `uint64_t` genügt für die Summe von höchstens 16 vorzeichenlosen 32-Bit-Werten. Die vorangehende Prüfung der Fensterlänge und des Zählerbetriebs bleibt eine separate Aufgabe der Messung.

```c
#include "lab07_summary.h"

int Lab07_Summarize(const volatile Lab07_Record *records, uint32_t count,
                    Lab07_Kind kind, Lab07_Summary *out)
{
    if (records == NULL || out == NULL || count > LAB07_CAPACITY ||
        (kind != LAB07_EMPTY && kind != LAB07_WORK)) {
        return 0;
    }
    Lab07_Summary s = {0};
    for (uint32_t i = 0U; i < count; ++i) {
        if (records[i].kind != (uint32_t)kind) { continue; }
        const uint32_t cycles = records[i].cycles_mod32;
        const uint32_t hz = records[i].core_hz;
        if (hz == 0U || (s.count != 0U && hz != s.core_hz)) {
            return 0;
        }
        if (s.count == 0U) {
            s.minimum = s.maximum = cycles;
            s.core_hz = hz;
        }
        if (cycles < s.minimum) { s.minimum = cycles; }
        if (cycles > s.maximum) { s.maximum = cycles; }
        s.sum += cycles;
        ++s.count;
    }
    if (s.count == 0U) { return 0; }
    *out = s;
    return 1;
}
```

Für den synthetischen 64-Bit-Fall ergeben sich Anzahl 2, Minimum 2000000000, Maximum 3000000000 und Summe 5000000000 Takte. Bei 160 MHz wäre der rechnerische Mittelwert 15,625 s. Diese großen Testzahlen dienen ausschließlich dazu, einen irrtümlichen 32-Bit-Summenüberlauf sichtbar zu machen. Für den Normalversuch werden die tatsächlich erfassten Werte verwendet.

Bei leerer gewählter Gruppe, ungültigem Argument, Nulltakt oder Taktabweichung in der gewählten Gruppe wird null zurückgegeben. Ein vorab mit bekannten Werten gefülltes `out` bleibt dabei unverändert. Ein Datensatz anderer Art zählt weder zur Summe noch zur Prüfung des Gruppentakts.

Bei der Umrechnung darf insbesondere das Produkt aus Anzahl und Frequenz nicht unbeabsichtigt in 32 Bit berechnet werden. Eine mögliche Auswertung nach dem Messfenster lautet `(double)s.sum / ((double)s.count * (double)s.core_hz)` für Sekunden. EMPTY und WORK werden zunächst separat beschrieben. Die LED bestätigt die erfolgreiche Analyse dieses Datensatzes, keine harte Frist und keine universelle WCET.

Quellen zur Einordnung: Buttazzo, *Hard Real-Time Computing Systems*; ST UM2408 und RM0399; Arm CMSIS. Alle Papierwerte und Funktionstestdaten sind ausdrücklich Lehrannahmen.